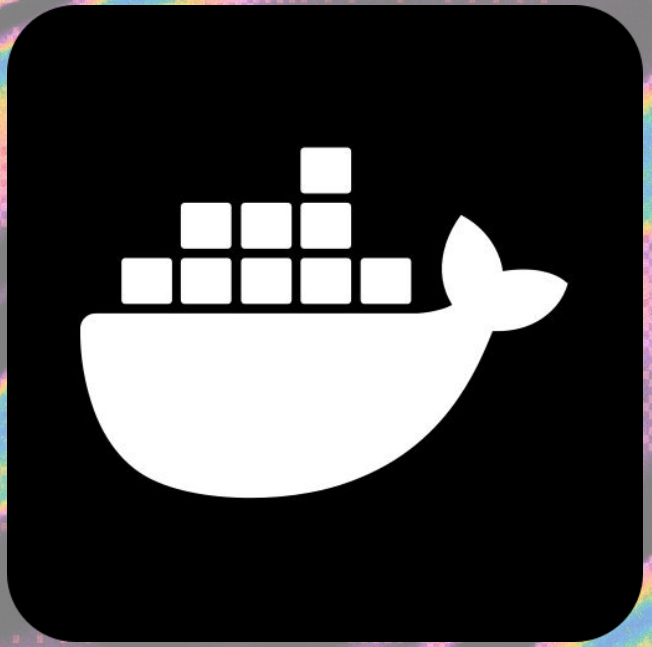




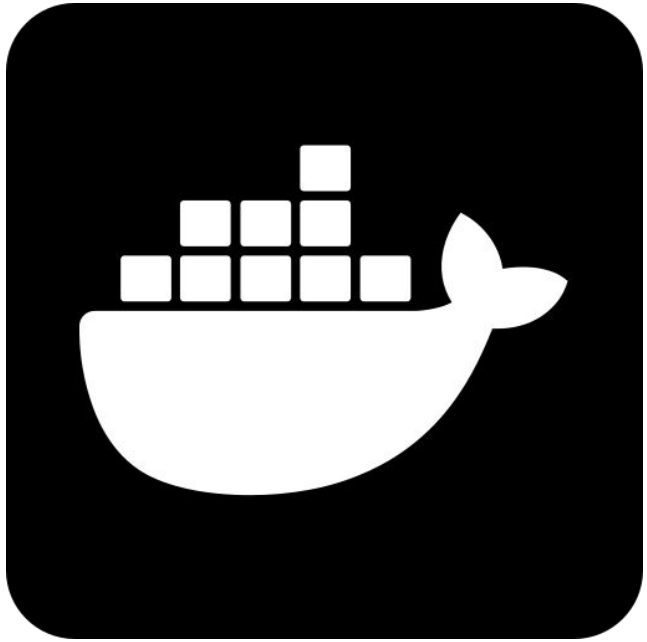


INTRO TO DOCKER

Presented by Walker Caskey (@DSiENT), Cyber Jedis Administrative Director & D5-Interactive Director

The Problem Docker Solves

- “Well it works on my machine!”
- Different OS versions, Missing libraries, Conflicting dependencies
- Virtual machines are HEAVY, each requires a full OS



What Is Docker?

Docker is an open platform for developing, shipping, and running applications in containers.

- Packages your app and its dependencies into one portable unit
- Runs the same way on your laptop, a lab machine, or the cloud
- “Build once, run anywhere”

What Is a Container?

Container = a runnable instance of an image

- Isolated from other containers and the host
- Shares the host OS kernel → lightweight
- Contains everything needed to run the app

Image vs Container vs Dockerfile

Image: Read-only template; includes OS base + your app

Container: A running (or stopped) instance of an image

Dockerfile: A text file with instructions to build an image

How Docker Is Used (Real-World)

- Development – consistent dev environments across a team
- CI/CD – build, test, and ship the same container
- Deployment – run on laptops, servers, or cloud with no changes
- Education – every student gets an identical environment

Docker Architecture (30-Second Version)

Client (docker CLI) → Daemon (dockerd) → Containers

- Client sends commands via REST API
- Daemon does the heavy lifting (build, run, distribute)
- Docker Desktop bundles everything for Windows/Mac

Essential Docker CLI Commands

`docker pull <image>`	Download an image from Docker Hub
`docker images`	List local images
`docker run <image>`	Create + start a container
`docker ps`	List running containers
`docker ps -a`	List all containers (including stopped)
`docker stop <container>`	Stop a running container
`docker rm <container>`	Remove a stopped container
`docker rmi <image>`	Remove an image
`docker logs <container>`	View container output
`docker exec -it <container> sh`	Open a shell inside a running container

Useful `docker run` **Flags**

`-d` → run in the background (detached)

`-p hostPort:containerPort` → publish a port so you can reach the app

`--name my-container` → give the container a friendly name

`-v vault_data:/app/notes` → attach a named volume (Docker manages the storage)

Example:

```
docker run -d -p 5000:5000 -v vault_data:/app/notes --name vault my-vault
```

Later we'll replace -p with a Tailscale sidecar for private HTTPS access.

Creating a Custom Image: The Dockerfile

A Dockerfile is a text file with build instructions.

Common instructions:

- `FROM` – base image
- `WORKDIR`– set working directory
- `COPY` – copy files from host into image
- `RUN` – execute a command during build
- `CMD` – default command to run when container starts

Example: Dockerfile for a Python Markdown Vault

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

RUN mkdir -p /app/notes

EXPOSE 5000

CMD ["shiny", "run", "--host", "0.0.0.0", "--port", "5000", "app.py"]
```

Docker Compose: Multi-Container Apps

Docker Compose defines and runs `multi-container apps` from one `YAML` file.

- Default file: compose.yaml (or compose.yml)
- One command to start/stop everything
- Perfect for “app + database” or “app + cache” setups

Compose can orchestrate networking sidecars like Tailscale for secure remote access.

A Minimal `compose.yaml` Example

```yaml
services:
  vault:
    build: .
    volumes:
      - vault_data:/app/notes
    expose:
      - "5000"
  caddy:
    image: caddy:2-alpine
    command: caddy reverse-proxy --from :80 --to vault:5000
    expose:
      - "80"
    depends_on:
      - vault
  tailscale:
    image: tailscale/tailscale:latest
    environment:
      - TS_CLIENT_ID=${TS_CLIENT_ID}
      - TS_CLIENT_SECRET=${TS_CLIENT_SECRET}
      - TS_EXTRA_ARGS=--advertise-tags=tag:container
      - TS_STATE_DIR=/var/lib/tailscale
      - TS_SERVE_CONFIG=/config/serve.json
    volumes:
      - tailscale_state:/var/lib/tailscale
      - ./serve.json:/config/serve.json:ro
    depends_on:
      - caddy
volumes:
  vault_data:
  tailscale_state:
```

Key Compose Commands

`docker compose up`	Build + start all services
`docker compose up -d`	Start in background
`docker compose down`	Stop + remove containers
`docker compose logs`	View logs from all services
`docker compose ps`	List services and status
`docker compose logs tailscale`	View logs from specific service in compose stack

Linux vs Windows: What Changes?

Docker Desktop on Windows runs Linux containers inside a lightweight VM (`WSL2` or `Hyper-V`).

- `WSL2` backend is recommended for most Windows 10/11 users
- Docker Desktop on Linux also runs a VM for consistency and security (Dont use docker desktop on linux - CLi only)

File-system gotcha

- Bind-mount from the Linux filesystem (inside `WSL2`) for best performance
- Avoid bind-mounting from C:\ → `docker run -v /mnt/c/…` is slow

Windows Containers vs Linux Containers

- Docker Desktop on Windows supports two modes:
  - Linux containers (default, most common)
  - Windows containers (for .NET Framework, Windows-specific workloads)
- You can switch modes, but not run both simultaneously
- Windows Home/Education editions only support Linux containers

Capstone: A Drag-and-Drop Markdown Vault

- Backend: Minimal python `shiny-express` web server
- Frontend: Clean web GUI listing notes
- Upload: Drag-and-drop .md files into the upload control
- Tailscale sidecar for tailnet access
- Caddy reverse proxy
- Named volume `vault_data`

Live Demo: Build & Run the Vault

Now we will begin the demo!

Key Takeaways & Next Steps

- Docker packages apps into portable, reproducible containers
- Images are templates; containers are running instances
- Dockerfiles build custom images; Compose orchestrates multi-container apps
- *Windows users:* use WSL2 and keep projects inside the Linux filesystem for speed
- Tailscale provides zero-config private networking with automatic HTTPS.
- OAuth clients give Tailscale containers permanent credentials; no 90-day rotation.

Resources

- Official docs: docs.docker.com
- CLI cheat sheet: docs.docker.com/get-started/docker_cheatsheet.pdf
- Control docker with python: gabrieldemarmiesse.github.io/python-on-whales/
- Shiny Express: shiny.posit.co/py/api/express/
- Tailscale Docker docs: tailscale.com/kb/1282/docker